

Closure

A closure is like a **bank locker**.

Even after you leave the bank,
your locker still holds your valuables.

Function is gone.
But memory is preserved.

Closure is a feature in JavaScript where **an inner function remembers and can access variables from its outer (lexical) scope even after the outer function has finished executing**.

```
function outer() {  
  let count = 0;  
  
  return function inner() {  
    count++;  
    console.log(count);  
  };  
}  
  
const counter = outer();  
  
counter(); // 1  
counter(); // 2  
counter(); // 3
```

What's Happening Internally?

Even though `outer()` has finished execution:

- inner() still has access to count
- JavaScript keeps count alive in memory
- That preserved environment = **Closure**

Give a Practical Use Case of Closures

Data Privacy / Encapsulation

```
function createBankAccount(balance) {  
  return {  
    deposit(amount) {  
      balance += amount;  
    },  
    getBalance() {  
      return balance;  
    }  
  };  
}
```

```
const account = createBankAccount(1000);  
account.deposit(500);  
console.log(account.getBalance()); // 1500
```

balance is private → accessible only via closure.

Can Closures Cause Memory Leaks?

Yes.

If closures hold references to unused large objects, GC cannot clean them.

Important in:

- Event listeners
- Long-lived callbacks

What Gets Stored in a Closure?

Not the variables themselves, but **references to the lexical environment**.

That's why values can change.

Is Closure Created Every Time a Function is Created?

 Yes.

Every function in JavaScript forms a closure with its surrounding scope.